SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I T NITHIN KUMAR (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: NITHIN KUMAR

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

SCENARIO BASED ASSIGNMENT – ANSWERS

Artificial Intelligence (Course Code: CSA17)

Course Outcome CO2 – Search and Game-Solving Techniques (Greedy Search, A*, CSP, Minimax, Alpha-Beta Pruning)

Contents of this report

Section	Scenario	Main techniques
1	AI-Powered Emergency Medicine Delivery Drones	Greedy Best-First Search, A*, heuristics, re-planning
2	Smart City Traffic Signal Optimization	Optimization, Hill Climbing, Simulated Annealing, Genetic Algorithms
3	Autonomous Mars Exploration Rover	Online search, partial observability, LRTA*, D* Lite
4	University Examination Timetable Generation	CSP, backtracking, forward checking, arc consistency
5	Strategic Game AI (Minimax and Alpha-Beta Pruning)	Minimax, Alpha-Beta pruning, evaluation functions, iterative deepening
6	Overall Conclusion and References	Summary of recommendations

Each scenario is answered in the order of the original questions: Scenario Overview, Objective, Detailed Answers (Tasks i–v, with technical explanation, worked example and diagram), and Conclusion. Numerical values in the examples are small illustrative values chosen to explain the algorithms; they are not experimental measurements.

1. SCENARIO 1: AI-POWERED EMERGENCY MEDICINE DELIVERY DRONES

1.1 Scenario Overview

A government uses AI-powered drones to deliver emergency medicines in a flood-affected region. The environment changes quickly: roads and air corridors can become unusable without warning, and weather and terrain change the real cost of travelling between two places. The drone knows a heuristic estimate (the straight-line distance to the destination), receives only partial real-time updates about blocked paths, and faces variable movement costs. Because the medicines are urgent, the drone must choose a route quickly while keeping delay and risk as low as possible.

1.2 Objective

To compare Greedy Best-First Search and A* Search for route planning in this uncertain, dynamic setting, to analyse how heuristic quality and blocked paths affect each algorithm, and to recommend the algorithm that best balances speed, route quality (optimality) and safety.

1.3 Detailed Answers

1.3.0 Problem formulation used in the answers

- **States:** waypoints (landing points, junctions, relief camps) in the flood region.
- **Actions:** fly along a corridor from one waypoint to a neighbouring waypoint.
- **Step cost *c(n, n')*:** distance × terrain/weather factor + risk penalty. A flooded or stormy corridor therefore costs more than a clear one even if it is shorter.
- **Heuristic *h(n)*:** straight-line distance from *n* to the destination.
- **Goal test:** the drone is at the relief camp.

Assumption: the cost factors are normalised so that the cost of moving one unit of distance is never less than one unit. Under this assumption the straight-line distance never overestimates the true remaining cost, so *h(n)* is **admissible**. All numerical values used below are small illustrative numbers chosen to explain the algorithms; they are not measured data.

1.3.1 Task (i): Behaviour of Greedy Best-First Search, with strengths and weaknesses under uncertainty

Technical explanation. Greedy Best-First Search always expands the frontier node that *appears* closest to the goal. Its evaluation function uses only the heuristic and ignores the cost already spent:

$$f(n) = h(n)$$

Working procedure:

1. Put the start node in a priority queue ordered by $h(n)$.
2. Remove the node with the smallest $h(n)$; if it is the goal, stop.
3. Generate its successors, skip nodes already explored, and insert the rest into the queue.
4. Repeat. The drone therefore keeps "heading straight for the destination".

Behaviour in the flood scenario. The drone prefers the waypoint that is physically nearest to the relief camp. When the corridor is clear this gives a decision almost immediately, because only one number per node is needed. But the straight-line estimate does not know that a short corridor over deep floodwater, strong wind or rough terrain is expensive or risky, so the drone can be pulled into a poor route.

Table 1.1: Strengths and weaknesses of Greedy Best-First Search under uncertainty

Strengths	Weaknesses
Very fast decisions: few nodes are expanded when the heuristic is informative.	Ignores the cost already spent $g(n)$, so it does not consider weather or terrain cost.
Low computation and memory use, suitable for a small on-board processor.	Not optimal: it can return a long or costly route (see the example below).
Easy to restart after a new blocked-path report, because it only needs the current node and h.	Can be trapped by dead ends or blocked roads that point towards the goal; may need to backtrack far.
Gives a usable "first route" very early (an anytime-style fallback).	With inaccurate or misleading estimates it can wander; in the worst case it behaves like depth-first search (time $O(b^m)$). It is complete only if repeated states are checked in a finite graph.

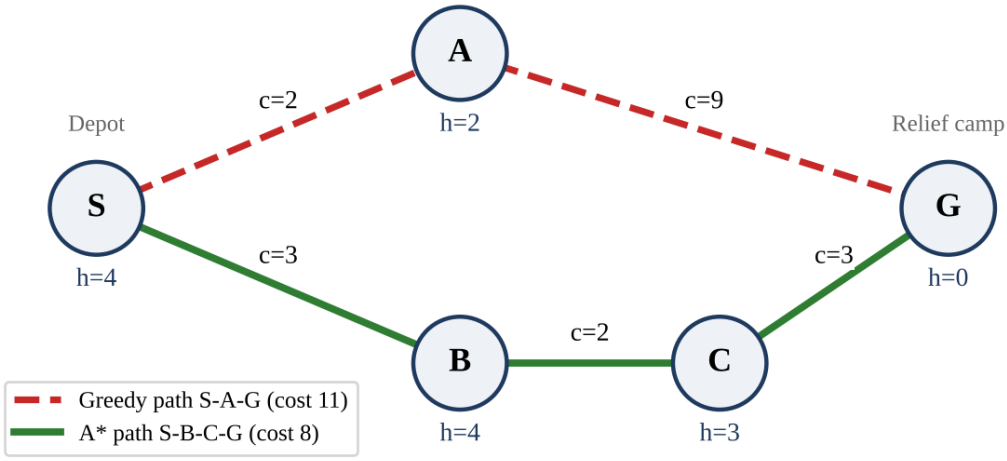


Figure 1.1: Example route graph. Edge labels are step costs c; node labels h are straight-line estimates to G. Greedy follows the dashed red path; A finds the solid green path.*

Worked example (Figure 1.1). From S, Greedy compares $h(A) = 2$ and $h(B) = 4$, so it goes to A, and then to G because $h(G) = 0$. The route S→A→G costs 2 + 9 = **11**, because the road A→G crosses badly flooded terrain (cost 9). The cheaper route S→B→C→G costs 3 + 2 + 3 = **8**, but Greedy never considers it. This shows how the algorithm can look attractive and still produce a higher-cost, higher-risk route.

1.3.2 Task (ii): How A handles the situation and balances cost and heuristic*

Technical explanation. A* evaluates each frontier node with the sum of the real cost already paid and the estimated cost still to pay:

$$f(n) = g(n) + h(n)$$

- $g(n)$ = actual cost from the depot to node n (distance, terrain, weather and risk already included).
- $h(n)$ = estimated cheapest cost from n to the relief camp (straight-line distance).
- $f(n)$ = estimated cost of the best complete route that passes through n.

Balancing cost and heuristic. $g(n)$ keeps the search honest: a node reached by an expensive route has a large f, so it falls behind in the queue. $h(n)$ gives direction: among nodes with similar g, those nearer the goal are expanded first. When $h = 0$ A* behaves like Uniform-Cost Search (careful but slow); when g is ignored it behaves like Greedy (fast but careless). A* uses both.

Optimality condition. If $h(n)$ is admissible (never overestimates) A* returns a least-cost route in tree search; if $h(n)$ is also consistent, $h(n) \leq c(n, n') + h(n')$, the first time a node is expanded

its *g* value is already optimal, so no re-expansion is needed. The heuristic in Figure 1.1 is consistent.

Table 1.2: A* trace on Figure 1.1 (nodes expanded in order)

Step	Node expanded	g	h	f = g + h	Frontier after expansion (node: f)
1	S	0	4	4	A: 2+2 = 4; B: 3+4 = 7
2	A	2	2	4	B: 7; G (via A): 11+0 = 11
3	B	3	4	7	C: 5+3 = 8; G (via A): 11
4	C	5	3	8	G (via C): 8+0 = 8 replaces G (via A): 11
5	G	8	0	8	Goal reached: route S→B→C→G, cost 8

A* first explores A (because *f(A) = 4* looks promising), then discovers that the road A→G is expensive, and finally finds the cheaper route of cost 8. In a weather-affected region this ability to correct an early promising-looking choice is the main advantage over Greedy. The price is memory: A* must store all generated nodes, and in the worst case time and memory grow exponentially.

1.3.3 Task (iii): Impact of heuristic accuracy on both algorithms

Table 1.3: Effect of heuristic quality

Heuristic quality	Effect on Greedy Best-First Search	Effect on A*
Very accurate (close to true remaining cost, never above it)	Picks the right neighbour most times; very fast and often near-optimal, but still no guarantee.	Expands very few nodes; the optimal route is found with little search.
Admissible but weak (e.g. straight-line distance that ignores weather)	Misleads the drone into short but costly or risky corridors; long detours when a road is blocked.	Still optimal, but it expands more nodes because *h* underestimates; slower but safe.
Zero (h = 0)	Behaves randomly (no guidance).	Becomes Uniform-Cost Search: optimal but explores in all directions.
Overestimating (inadmissible)	Little difference, because Greedy never guarantees optimality.	Faster, but optimality is lost: a good route may be pruned away.
Perfect (h = true cost)	Goes straight to the goal along an optimal route.	Expands only nodes on an optimal route.

- **For Greedy,** heuristic accuracy decides the *quality of the route* as well as the speed, because the heuristic is the *only* information used.
- **For A*,** heuristic accuracy affects mainly the *amount of search*; the route stays optimal as long as the heuristic is admissible. Greater accuracy means fewer expanded nodes.

- **Weather-aware improvement.** Straight-line distance is accurate for distance but not for cost. A better heuristic is $h'(n) = d(n, goal) \times c_{min}$, where c_{min} is the lowest cost per unit distance that is currently possible in the region. It is closer to the true cost yet remains admissible, as long as c_{min} is never larger than the real minimum.
- **Risk of inaccurate updates.** If real-time reports are wrong (a road reported open that is actually flooded) any heuristic that uses them can become inadmissible, so safety margins on the cost estimates are needed.

1.3.4 Task (iv): Effect of dynamic changes (blocked paths) on search performance

A blocked path removes an edge, or increases its cost, *after* a plan was made. The effect on each algorithm is shown below.

Table 1.4: Effect of blocked paths and changing costs

Aspect	Greedy Best-First Search	A*
When the planned path becomes blocked	Move to the next-best node by *h*; may have to backtrack out of a dead end towards the goal.	Previous *g* and *f* values are outdated; the plan must be recomputed from the current position.
Cost of recovering	Cheap per step, but the route can become much longer.	More computation per re-plan; the new route is again optimal for the updated map.
Partial information	Does not use costs, so it cannot tell that the new route is more dangerous.	Uses updated costs, so a risky corridor automatically becomes less attractive.
Performance trend	Fast but unstable; quality drops with each change.	Stable quality; delay depends on how often and how far the plan changes.

Practical remedies. (1) Re-plan from the drone's *current* position whenever a relevant update arrives (Figure 1.2). (2) Use incremental re-planning methods such as D* Lite or Lifelong Planning A*, which reuse earlier search effort and update only the affected part of the graph. (3) Re-plan only within a limited horizon when the drone is far from the destination. (4) Keep a safe fallback (return to base or nearest safe landing point) if no route remains.

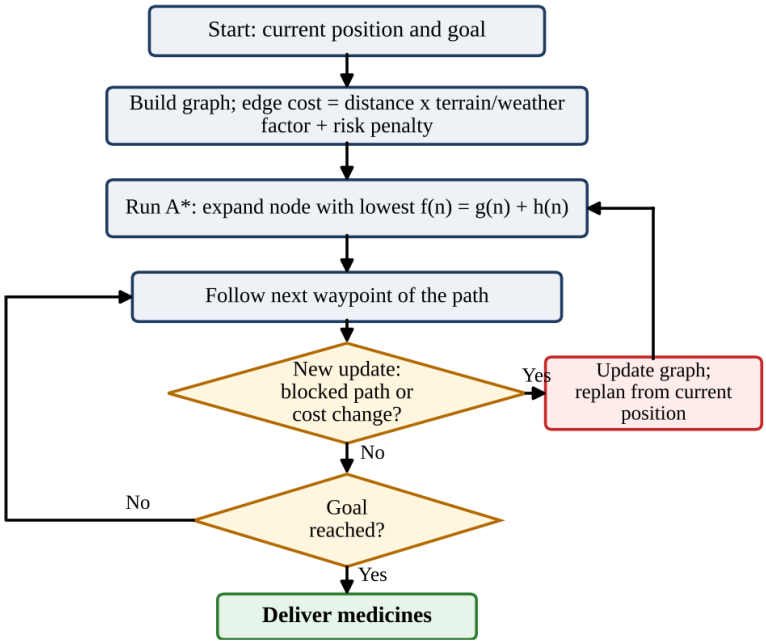


Figure 1.2: A route planning with re-planning whenever a blocked path or cost change is reported.*

1.3.5 Task (v): Recommended algorithm and justification

Recommendation: A* Search with an admissible, weather-aware heuristic and re-planning on every important update (preferably in an incremental form such as D* Lite). When the time limit is very tight, use Weighted A*, $f(n) = g(n) + w \cdot h(n)$ with w slightly above 1, which is faster and guarantees a cost not greater than w times the optimum when h is admissible; Greedy search is kept only as a last-resort fallback.

Table 1.5: Justification against the three decision criteria

Criterion	Greedy Best-First	A* (recommended)
Real-time constraints	Fastest per decision, but may need many corrections later.	Slower than Greedy, but adequate for a graph of waypoints; Weighted A* and D* Lite reduce delay.
Optimality	Not optimal.	Optimal with an admissible heuristic (bounded sub-optimality with Weighted A*).
Safety	Ignores terrain/weather/risk costs, so can choose dangerous corridors.	Risk can be included in the step cost, so dangerous corridors are avoided automatically.
Handling new information	Easy to continue, quality not controlled.	Re-plan from the current position using the updated costs.

A* is the better choice because a slightly slower decision is acceptable, while a route through an unsafe corridor is not. The extra time is controlled by limiting the search horizon, by using Weighted A* when necessary and by incremental re-planning.

1.4 Conclusion

Greedy Best-First Search is fast and simple but ignores the cost already incurred, so it can be misled by heuristics and by blocked paths and cannot guarantee a safe or optimal route. A* combines the actual cost *g(n)* with the estimate *h(n)*, remains optimal under an admissible heuristic and can include weather and risk in its costs. For emergency drone delivery, A* with re-planning (and Weighted A* when time is critical) is recommended because it best satisfies real-time, optimality and safety requirements. Its guarantees depend on the assumptions stated above: an admissible heuristic and sufficiently accurate cost updates.

2. SCENARIO 2: SMART CITY TRAFFIC SIGNAL OPTIMIZATION

2.1 Scenario Overview

A smart city uses an AI system to set the signal timings of hundreds of intersections. The system must reduce total waiting time, fuel consumption and congestion. Traffic flow changes continuously, so the signals must be adjusted continuously. The number of possible timing combinations is extremely large, and a simple search method may stop at a solution that is only locally good.

2.2 Objective

To formulate traffic-signal control as an optimization problem, explain why traditional search is inefficient, analyse Hill Climbing and its limitations, show how Simulated Annealing and Genetic Algorithms overcome them, and recommend the approach that is most scalable and adaptable.

2.3 Detailed Answers

2.3.0 Formulation as an optimization problem, and why traditional search is inefficient

Decision variables. For every intersection i ($i = 1 \dots N$): the cycle length C_i, the green time of each phase $g_{i,k}$ and the offset θ_i (the delay between its cycle start and that of its neighbour, used to create "green waves"). A candidate solution x is the vector of all these values.

Objective function (to be minimised).

$$F(x) = w_1 \cdot W(x) + w_2 \cdot E(x) + w_3 \cdot K(x)$$

- $W(x)$ = total vehicle waiting time (sum of delay of all vehicles over a period).
- $E(x)$ = fuel consumption, estimated from idling time and the number of stop-and-go events.
- $K(x)$ = congestion measure, for example the sum of queue lengths divided by the storage capacity of each road link.
- w_1, w_2, w_3 = weights chosen by the city (for example $w_1 + w_2 + w_3 = 1$) after each term is scaled to a comparable range.

Constraints.

- $g_{min} \leq g_{i,k} \leq g_{max}$ (minimum green for safety and pedestrians, maximum green to prevent starvation of other roads).

- $\sum_k g_{i,k} + L_i = C_i$, where L_i is lost/clearance time, and $C_{min} \le C_i \le C_{max}$.
- $0 \le \theta_i < C_i$; conflicting movements are never green at the same time.
- Queue spill-back and emergency-vehicle priority limits must be respected.

Size of the search space. If each intersection has only 20 candidate timing plans, 100 intersections give $20^{100} \approx 10^{130}$ combinations, far more than can be enumerated. Real systems have hundreds of intersections and continuous variables, so the space is even larger.

Table 2.1: Why traditional (uninformed/informed) search is inefficient

Reason	Explanation
No clear goal state	Traditional search looks for a path to a goal. Here there is no single goal; the aim is to find the best configuration among many.
Enormous space	BFS, DFS and Uniform-Cost Search enumerate states, so time and memory grow exponentially with the number of intersections.
Expensive evaluation	Every candidate must be evaluated with a traffic model or simulation, so each node is costly.
Strong interaction	Changing one junction affects its neighbours (queues and platoons), so the problem cannot be split into independent searches.
Changing environment	The best solution changes with traffic flow, so a complete search result becomes outdated before it finishes.
Path is irrelevant	Only the final configuration matters, not the sequence of steps; this is the setting for *local search* and population-based methods.

2.3.1 Task (i): Working of Hill Climbing and its limitations

Technical explanation. Hill Climbing is a local search algorithm. It keeps one current solution, examines its neighbours and moves to a neighbour that has a better objective value. A neighbour is created by a small change, for example increasing one phase's green time by 5 seconds or shifting one offset by 3 seconds. Because lower F is better, the search moves "uphill" on $J(x) = -F(x)$.

```
HILL-CLIMBING (steepest ascent)
  current ← initial timing plan (e.g. current plan or a fixed-time plan)
  loop:
    neighbours ← plans obtained by one small change in current
    best ← neighbour with the lowest F
    if F(best) ≥ F(current): return current      // no improving move
    current ← best
```

Working in this scenario. Start from the current plan, simulate several small changes, keep the one that reduces waiting time, fuel and congestion most, and repeat. Advantages: very simple, little memory, quick improvement, can run continuously.

Limitations:

- It stops at the first solution that has no better neighbour, which may be far from the best.
- It cannot escape flat areas (no gradient to follow) and moves slowly or randomly there.
- It struggles when several variables must change together (ridges).
- The final result depends heavily on the starting plan; it is not complete and not optimal.
- Every neighbour requires an evaluation, which is expensive when there are hundreds of intersections.

2.3.2 Task (ii): Local maxima, plateaus and ridges with real-world interpretation

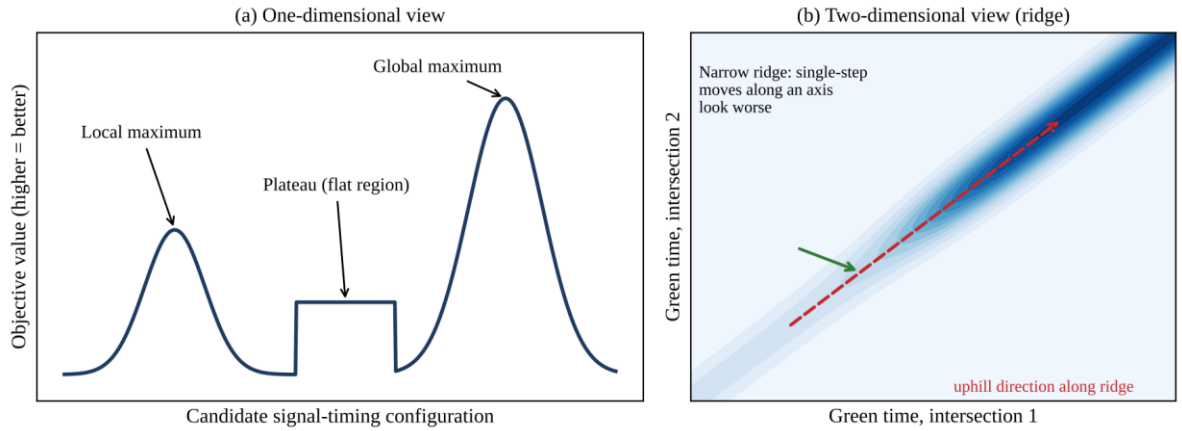


Figure 2.1: (a) Local maximum, plateau and global maximum on a one-dimensional objective curve; (b) a ridge on a two-variable objective surface.

Table 2.2: Hill-climbing difficulties and their traffic interpretation

Problem	Meaning	Traffic example
Local maximum	A solution better than all neighbours but worse than the global best.	A coordinated green wave on one main road is the best that single-junction changes can achieve, but a different timing pattern for the whole corridor would be better. Any single change makes things worse, so the algorithm stops.
Plateau	A region where neighbouring solutions have the same objective value, so there is no direction to move.	At a lightly loaded junction, changing green time by a few seconds does not change total waiting time at all, so the algorithm cannot tell which direction to go.
Ridge	A narrow high region where improvement needs several variables to change together; single-variable moves look worse.	To keep a platoon of vehicles moving through two junctions, the green time of junction 1 *and* the offset of junction 2 must both change. Changing only one breaks the coordination and increases waiting.

These three situations explain why the statement “the system may get stuck in locally optimal solutions” is true for traffic networks, where neighbouring junctions are strongly coupled.

2.3.3 Task (iii): How Simulated Annealing or Genetic Algorithms overcome these limitations

A. Simulated Annealing (SA)

Technical explanation. SA behaves like Hill Climbing, but it sometimes accepts a *worse* neighbour. The probability of accepting a move that increases the cost by $\Delta F = F(new) - F(current) > 0$ is

$$P(accept) = e^{-\Delta F / T}$$

where T is the "temperature". A move that improves the cost ($\Delta F \leq 0$) is always accepted. T starts high and is reduced gradually (cooling), for example $T_{k+1} = \alpha T_k$ with α between 0 and 1 (such as 0.95).

Numerical illustration. For a worsening move with $\Delta F = 4$: at $T = 10$ the acceptance probability is $e^{-0.4} \approx 0.67$; at $T = 2$ it is $e^{-2} \approx 0.14$. Early in the search the algorithm explores freely and can leave a local maximum; later it behaves almost like Hill Climbing.

```
SIMULATED-ANNEALING
  current ← initial plan;  T ← T0
  while T > T_min:
    next ← random neighbour of current;   ΔF ← F(next) − F(current)
    if ΔF ≤ 0 or random() < exp(−ΔF / T): current ← next
    keep best plan found so far;   T ← α · T
  return best plan
```

B. Genetic Algorithm (GA)

Technical explanation. A GA keeps a *population* of complete timing plans and improves it over generations.

- **Chromosome:** the list of green times and offsets for all intersections (or all intersections of a zone).
- **Fitness:** $fitness(x) = 1 / (1 + F(x))$, so a lower cost gives a higher fitness.
- **Selection:** better plans are more likely to be chosen as parents (tournament or roulette-wheel selection).
- **Crossover:** two parents exchange segments, for example the timings of whole neighbouring corridors, which preserves good coordinated blocks.
- **Mutation:** small random changes to a gene add diversity.
- **Elitism:** the best plans are copied unchanged so that quality is never lost.

Table 2.3: How SA and GA address each Hill-Climbing limitation

Limitation	Simulated Annealing	Genetic Algorithm
Local maximum	Accepts worse moves with probability $e^{-\Delta F/T}$, so it can climb out.	Population holds many different plans; mutation and crossover jump to new regions.
Plateau	Random moves continue to explore the flat region and can find its edge.	Different individuals sit in different places, and selection favours any that find an exit.
Ridge	Occasional downhill moves let it step across to the ridge direction.	Crossover can change several variables at once, moving along the ridge.
Dependence on start	Less dependent when starting temperature is high.	Starts from many points at once.
Weakness	Needs a good cooling schedule; sequential by nature.	Needs many evaluations and parameter tuning; no guarantee of the global optimum.

2.3.4 Task (iv): Recommended approach, justified by scalability and adaptability

Recommendation: a hybrid in which a Genetic Algorithm performs the global search and Simulated Annealing (or a short hill-climbing phase with annealing-style acceptance) refines the best plans locally. The city is divided into zones; each zone is optimised with the same method, and neighbouring zones exchange boundary timings so that corridors stay coordinated.

Table 2.4: Comparison of methods (qualitative)

Criterion	Hill Climbing	Simulated Annealing	Genetic Algorithm	Hybrid GA + SA
Escapes local maxima	No	Yes	Yes	Yes
Scalability	Good (cheap per step)	Good	Good with parallel evaluation and zones	Good (zones, parallel)
Adaptability to new traffic	Needs restart; may stay stuck	Can restart from current plan	Population can be re-seeded with the current plan	Warm start + fast local repair
Evaluation cost	Low	Medium	High	Medium–high, controllable
Optimality guarantee	None	None in practice	None	None

- **Scalability:** the fitness of many individuals can be computed in parallel; zone decomposition keeps each chromosome short; SA needs only one evaluation per step.
- **Adaptability:** when traffic changes, the population or the previous best plan is used as the starting point (warm start), so the system reacts quickly instead of beginning again; high temperature can be briefly restored to re-explore.

- **Quality and safety:** the constraint checks (minimum/maximum green, pedestrian time) are applied before any plan is deployed.

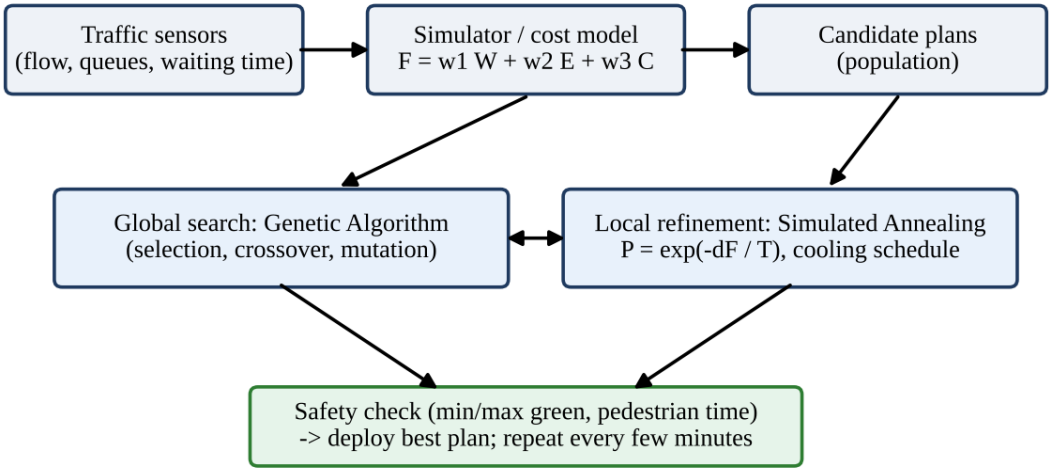


Figure 2.2: Recommended hybrid optimisation loop for city-wide signal control.

Note on guarantees: none of these methods guarantees the global optimum. They give good solutions within a limited time, which is what a real-time traffic controller needs.

2.4 Conclusion

Traffic-signal control is an optimization problem with a huge search space, costly evaluation, strongly coupled variables and a changing environment, so classical goal-directed search is unsuitable. Hill Climbing is simple and fast but is trapped by local maxima, plateaus and ridges. Simulated Annealing escapes by accepting worse moves with a decreasing probability, and Genetic Algorithms escape by exploring many solutions in parallel and recombining them. A hybrid GA–SA approach applied zone by zone, with warm starts and safety checks, is recommended for its scalability and adaptability.

3. SCENARIO 3: AUTONOMOUS MARS EXPLORATION ROVER

3.1 Scenario Overview

An autonomous rover explores unknown terrain on Mars. It must navigate safely to collect samples, avoid hazards such as craters and rocks, work with limited sensing capability and make decisions without a complete map. As it explores, it updates its knowledge, so planning and acting happen together in real time.

3.2 Objective

To explain why an online search agent is required, analyse the effects of partial observability and a dynamic environment, describe the rover's internal model, compare online search with uninformed and informed search, and justify the most suitable approach in terms of uncertainty, adaptability and safety.

3.3 Detailed Answers

3.3.1 Task (i): Why an online search agent is needed instead of offline search

Technical explanation. An **offline** search agent computes a complete plan (a full sequence of actions) *before* acting, using a known model of the environment. An **online** search agent **interleaves** computation and action: it chooses an action, executes it, observes the result and then decides the next action. Online search is the natural choice when the environment is unknown or changes while the agent acts.

Table 3.1: Offline versus online search for the Mars rover

Factor	Offline search	Online search (needed here)
Knowledge of the map	Requires a complete, correct map beforehand.	Works with an unknown or incomplete map and discovers it while moving.
Planning time	Entire plan computed first; may be wasted if the map is wrong.	Plans only the next step(s) and uses new observations immediately.
Hazards found during travel	A new rock or crater makes the plan invalid.	A new hazard simply changes the next decision.
Communication with Earth	A plan could be sent from Earth, but one-way signal delay is several minutes (roughly 3 to 22 minutes depending on planet positions), so Earth cannot react quickly to new hazards.	Decisions are made on board, in real time.
Risk of irreversible loss	A wrong map may lead the rover into a dead end or crater.	Moves are cautious; each step is checked with fresh sensor data.

Assumption: online search algorithms normally assume that the environment is *safely explorable*, meaning that the rover can always get out of any state it reaches. A rover must preserve this property by avoiding irreversible moves, such as driving into very steep slopes or loose sand.

3.3.2 Task (ii): Challenges of partial observability and dynamic environments

Table 3.2: Challenges and mitigation

Challenge	Why it occurs	Effect on the rover	Mitigation
Limited sensing range and occlusion	Cameras/sensors see only nearby terrain; rocks hide what is behind them.	The rover does not know what is beyond the visible area; it may enter a hidden hazard.	Treat unknown cells as uncertain; move in small steps; scan before moving.
Noisy sensors and localisation drift	Wheel slip, dust and lighting errors affect measurements.	Map and position estimates become inaccurate.	Keep probabilities rather than yes/no values; combine odometry, inertial and visual data; correct drift using landmarks.
Changing terrain	Dust storms, shifting sand, falling rocks, changing shadows and temperature.	A previously safe cell may become unsafe.	Re-observe and re-plan regularly; reduce confidence of old observations.
Limited resources	Battery, time, wheel wear, thermal limits.	Long detours may be impossible.	Include energy in the cost; keep a reserve; return to safe areas.
Several possible world states (belief state)	Because the rover cannot see everything it can be in many possible states.	Plans that assume one state may fail.	Plan conservatively; choose actions that work for the likely states.

3.3.3 Task (iii): How the rover builds and updates its internal model

Technical explanation. The internal model is the rover's memory of the world. It has these parts:

- **Map / hazard map:** a grid of cells, each marked unknown, free or hazardous with an associated probability.
- **Pose estimate:** the rover's own position and orientation, from odometry and visual/inertial data.
- **Result table *RESULT(s, a)*:** what state resulted when action *a* was taken in state *s*, learned from experience.
- **Cost-to-goal estimates *H(s)*:** the agent's current estimates of the cost to reach the target from each visited state, improved with experience.
- **Visit counts and resource state:** which cells have been visited and how much battery remains.

Update cycle (repeated after every move):

1. Sense the surroundings with cameras and range sensors.
2. Update the position estimate and add newly seen cells to the map.
3. Update each observed cell's hazard probability (Bayesian update below).
4. Recompute the cost of moving through each cell, adding a penalty for hazard probability.
5. Update *H(s)* of the current state, record *RESULT(s, a)*, and then choose the next action.

Probabilistic occupancy update. For a cell with prior hazard probability p, the log-odds $l = ln(p / (1 - p))$ is updated by adding the log-odds of each new observation:

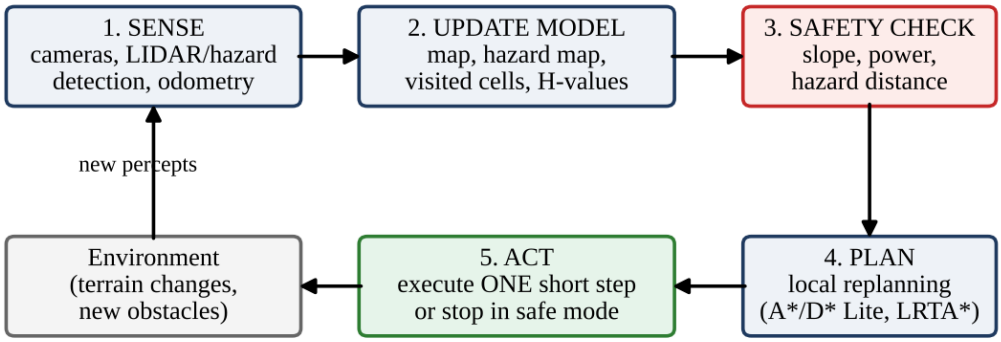
$$l_t(cell) = l_{t-1}(cell) + ln(p(cell \mid z_t) / (1 - p(cell \mid z_t)))$$

Example. A cell starts unknown ($p = 0.5$, $l = 0$). A sensor reports "hazard" with $p(cell \mid z) = 0.8$, giving $l = ln\ 4 \approx 1.39$ and $p = 0.80$. A second consistent observation adds 1.39 again: $l \approx 2.77$, $p = 16/17 \approx 0.94$. The rover is now much more confident that the cell is dangerous and the planner gives it a very high cost. A single contradictory observation reduces l again, so the model can be corrected.

Learning cost estimates (LRTA*). The Learning Real-Time A* algorithm updates its estimate each time it leaves a state:

$$H(s) \leftarrow min_a [\ c(s, a, s') + H(s')\ ]$$

For example, if the rover stands in a pit with $H(s) = 1$ and all exits cost 1 but lead to states with $H = 2$, the update gives $H(s) = min(1 + 2, 1 + 2) = 3$. The estimate of the pit has risen, so the rover no longer believes it is "close to the goal" and will leave rather than get stuck in a local minimum.



The loop repeats after every step: the rover never commits to a long plan based on a map that it knows to be incomplete.

Figure 3.1: Online sense–model–plan–act loop of the rover.

3.3.4 Task (iv): Online search compared with uninformed and informed search

Uninformed algorithms (BFS, DFS, Uniform-Cost Search) and informed algorithms (Greedy, A*) are classic *offline* algorithms: they assume that the whole state space and its costs are known. Online algorithms (online DFS, LRTA*, D* Lite) decide while acting. The comparison is therefore about *when* and *how* the algorithm uses information. In practice an online agent may call an informed algorithm such as A* repeatedly as a local planner on its updated map.

Table 3.3: Comparison of search strategies for the rover

Property	Uninformed (BFS, DFS, UCS)	Informed (Greedy, A*)	Online (LRTA*, D* Lite, online DFS)
Needs complete map before starting	Yes	Yes	No
Uses goal-direction knowledge	No	Yes, through *h(n)*	Yes, through learned *H(s)* or heuristics
Handles new hazards	Must restart	Must restart or re-plan	Naturally, after each observation
Completeness / optimality	BFS complete, UCS optimal on a known map	A* optimal with admissible *h* on a known map	Complete in safely explorable finite spaces; not optimal in the first trip, improves with learning (LRTA*)
Time and memory	High (many nodes)	Lower than uninformed, still high for A*	Small, because only local information and a table of estimates are kept
Suitability for Mars	Poor: exhaustive and needs a map	Good as a planner on a known part of the map	Best: acts with incomplete information

3.3.5 Task (v): Most suitable approach considering uncertainty, adaptability and safety

Recommendation: an online, model-based replanning agent. The rover keeps a probabilistic map; it uses an informed planner (A* or the incremental version D* Lite) on the *currently known* map to reach the next frontier or sample site; it uses LRTA*-style learning of *H(s)* to escape traps; and a safety layer checks every move before it is executed.

Table 3.4: Justification

Requirement	How the recommended approach meets it
Uncertainty	A probabilistic map represents what is unknown; unknown or uncertain cells carry extra cost; plans are short and are repeated.
Adaptability	After every step the model is updated and the plan is recomputed (D* Lite reuses earlier work, so the update is cheap); learning prevents loops and dead ends.
Safety	Hazard cells are treated as impassable or very costly; moves are limited by slope, wheel-slip and power limits; if no safe move exists the rover stops in safe mode and waits for instructions; only reversible steps are taken.
Efficiency	Only the nearby map is replanned, so computation is small enough for on-board processors.

3.4 Conclusion

Because the Martian terrain is unknown, partially observable and changing, and because Earth cannot respond in time, the rover needs an online search agent that senses, updates its model, plans a short step and acts repeatedly. Uninformed and informed offline algorithms are valuable building blocks, but only an online, replanning agent with a probabilistic map, learned cost estimates and a safety layer satisfies the requirements of uncertainty, adaptability and safety. This approach does not guarantee the shortest route but it gives dependable behaviour, which is the primary goal for exploration.

4. SCENARIO 4: UNIVERSITY EXAMINATION TIMETABLE GENERATION

4.1 Scenario Overview

A university wants an AI system that automatically generates examination timetables. The timetable must avoid overlapping exams for any student, respect the limited number of exam halls, schedule certain subjects before others, honour faculty availability and provide special accommodations for some students. The difficulty grows quickly as the number of courses and students increases.

4.2 Objective

To formulate exam timetabling as a Constraint Satisfaction Problem (CSP), show how backtracking search and constraint propagation solve it, and recommend a strategy that scales to real university data.

4.3 Detailed Answers

4.3.0 Formulation as a Constraint Satisfaction Problem

A CSP is defined by three components: a set of **variables** X, a **domain** D_i of possible values for each variable and a set of **constraints** C that specify which combinations of values are allowed. A solution is a *complete* assignment of values to all variables that violates no constraint. Exam timetabling is closely related to graph colouring (courses are vertices, a shared student is an edge, time slots are colours), which is why it is computationally hard in general.

4.3.1 Task (i): Variables, domains and constraints

Table 4.1: Variables and domains

Variable	Meaning	Domain	Example
Slot(c)	Time slot of the exam of course *c*	{1, 2, …, *T*}: all slots in the exam period (e.g. 3 days × 2 sessions = 6 slots)	Slot(AI) ∈ {1, …, 6}
Hall(c)	Hall used for course *c*	Set of halls (or a combination of halls) that can be used	Hall(AI) ∈ {H1, H2, H3}
Invig(c)	Invigilators for course *c* (optional)	Staff available at the chosen slot	Invig(AI) ⊂ Staff

Table 4.2: Constraints

Constraint	Type	Formal statement	Explanation / example
C1: No overlap for a student	Binary (hard)	If courses c_1 and c_2 share a student: $\text{Slot}(c_1) \neq \text{Slot}(c_2)$	A student taking Maths and Physics cannot have both exams at the same time.
C2: Limited halls	Global (hard)	For each slot t and hall h: the total number of students assigned ≤ capacity(h), and a hall is used by at most one exam in a slot	A 120-seat hall cannot hold 150 students.
C3: Precedence	Binary (hard)	If c_1 must be before c_2: $\text{Slot}(c_1) < \text{Slot}(c_2)$	Maths must be scheduled before AI.
C4: Faculty availability	Unary (hard)	$\text{Slot}(c) \in$ Available(faculty of c)	A lecturer who is away on day 2 cannot be an invigilator on day 2.
C5: Special accommodations	Unary / global (hard)	Hall(c) ∈ accessible halls if the course has a student who needs them; extra time requires a longer session	A wheelchair user must have a ground-floor hall; a student with extended time needs a longer slot.
C6: Preferences (soft)	Soft	Minimise: a student having two exams on the same day, or exams on consecutive days	Violation is allowed but penalised.

Hard constraints (C1–C5) must be satisfied; soft constraints (C6) are optimised after a feasible timetable has been found. For the worked example below a small subset is used.

Small illustrative instance. Five courses: M (Maths), P (Physics), A (AI), D (DBMS), N (Networks), and three slots {1, 2, 3}. Shared students create the constraints M≠P, M≠A, P≠A, A≠D and D≠N (written as inequality of slots). Precedence: M must be before A (M < A). DBMS needs a special large hall that is available only in slots 2 and 3, so $D \in \{2, 3\}$. The Networks faculty is available only in slot 2, so $N \in \{2\}$. At most two exams may be held in a slot (hall limit). Figure 4.1 shows the constraint graph.

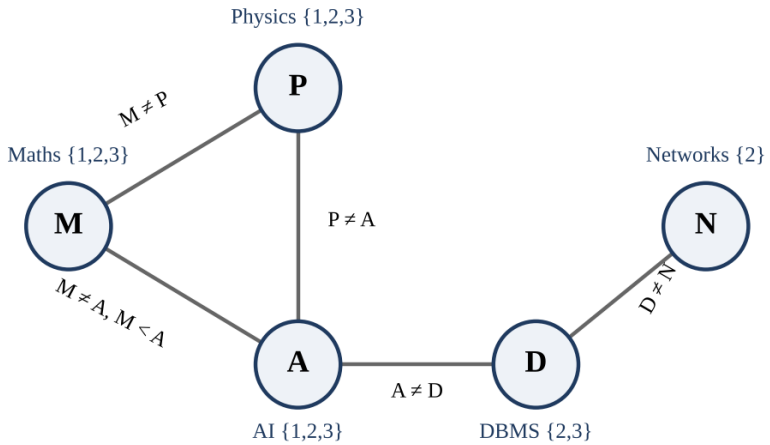


Figure 4.1: Constraint graph of the illustrative instance (domains shown beside each variable).

4.3.2 Task (ii): Backtracking search in this scenario

Technical explanation. Backtracking search is depth-first search over partial assignments. It assigns one variable at a time, checks every constraint involving assigned variables immediately, and if a variable has no legal value left it *backtracks* to the previous variable and tries its next value.

```
BACKTRACK(assignment):
  if assignment is complete: return assignment
  var ← SELECT-UNASSIGNED-VARIABLE        // e.g. MRV + degree heuristic
  for each value in ORDER-DOMAIN-VALUES(var):   // e.g. least-constraining value
    if value is consistent with assignment:
      add (var = value) to assignment
      result ← BACKTRACK(assignment)
      if result ≠ failure: return result
      remove (var = value) from assignment    // backtrack
  return failure
```

Step-by-step trace (plain backtracking, order M, P, A, D, N, values tried in increasing order):

Table 4.3: Backtracking trace

Step	Assignment tried	Result
1	M = 1	Consistent.
2	P = 1	Violates M ≠ P → reject.
3	P = 2	Consistent.
4	A = 1, then A = 2	A = 1 violates M ≠ A; A = 2 violates P ≠ A → reject both.
5	A = 3	Consistent (also M < A).
6	D = 2	Consistent (D ∈ {2, 3}, A ≠ D).
7	N = 2	Violates D ≠ N, and N has no other value → **backtrack**.
8	D = 3	Violates A ≠ D (A = 3) → D has no value left → backtrack.
9	Back at A	A had only the value 3 → backtrack to P.
10	P = 3	Consistent.
11	A = 1, then A = 2	A = 1 violates M ≠ A; A = 2 is consistent.
12	D = 2, then D = 3	D = 2 violates A ≠ D; D = 3 is consistent.
13	N = 2	Consistent. **Solution: M = 1, P = 3, A = 2, D = 3, N = 2.**

Check: slot 1 has {M}, slot 2 has {A, N}, slot 3 has {P, D}, so every slot has at most two exams; M≠P, M≠A, P≠A, A≠D, D≠N hold; M < A holds (1 < 2); D = 3 and N = 2 are in their allowed slots.

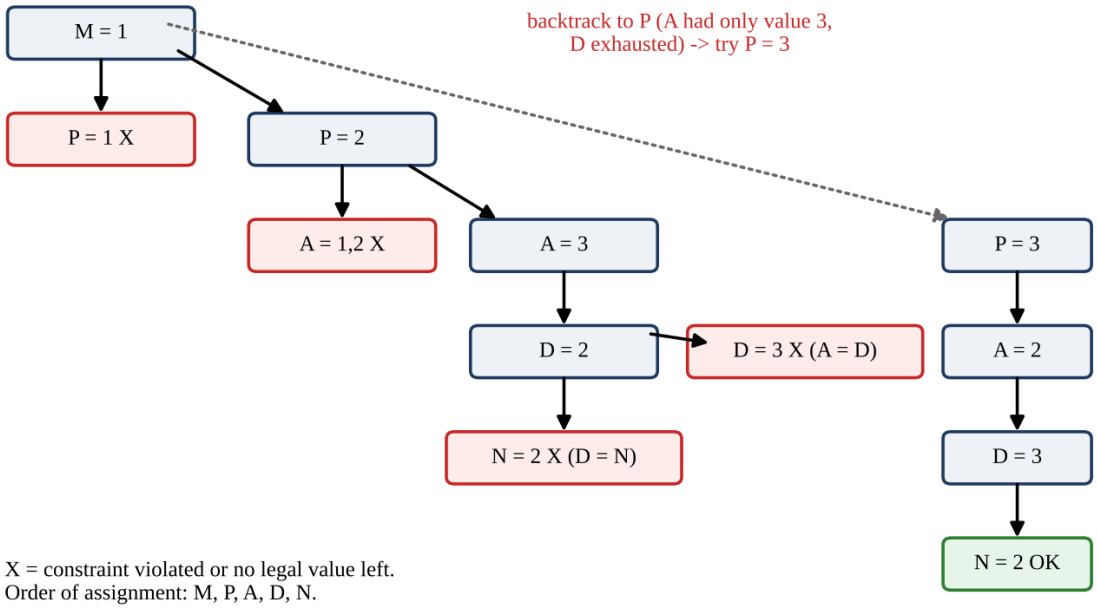


Figure 4.2: Backtracking search tree for the illustrative timetable (X marks a failed assignment).

In this trace plain backtracking tried **15** value assignments (counting each value tried, including rejected ones). The count is for this small example only. Good variable and value ordering heuristics reduce the work in larger problems:

- **Minimum Remaining Values (MRV):** choose the variable with the fewest legal values (here N, which has only one).
- **Degree heuristic:** among ties, choose the variable involved in most constraints (courses with many shared students).
- **Least-Constraining Value:** try first the value that rules out the fewest values for neighbours.

4.3.3 Task (iii): Constraint propagation and forward checking

Technical explanation. Constraint propagation uses the constraints to remove values from the domains of unassigned variables *before* they are tried. **Forward checking** is the simplest form: whenever a variable is assigned, every unassigned neighbour loses the values that conflict with it, and the search backtracks immediately if any domain becomes empty. **Arc consistency (AC-3)** goes further: it repeatedly removes a value from a variable's domain if some neighbour has no compatible value left, and it propagates this through the constraint graph. Maintaining arc consistency during search is called MAC.

Table 4.4: Forward checking on the illustrative instance

After assigning	M	P	A	D	N	Comment
(initial)	{1,2,3}	{1,2,3}	{1,2,3}	{2,3}	{2}	
M = 1	1	{2,3}	{2,3}	{2,3}	{2}	M ≠ P, M ≠ A and M < A remove 1 from P and A.
P = 2	1	2	{3}	{2,3}	{2}	P ≠ A removes 2 from A.
A = 3	1	2	3	{2}	{2}	A ≠ D removes 3 from D.
D = 2	1	2	3	2	{ }	D ≠ N empties N: **fail immediately**.

Forward checking never tries the doomed values P = 1, A = 1, A = 2 and D = 3 in the first branch, and finds the dead end as soon as D = 2 is chosen. With MAC the failure is detected even earlier: after P = 2 the domain of A becomes {3}, arc consistency then removes 3 from D (leaving {2}), and then removes 2 from N, which empties N, so the branch P = 2 is rejected before A and D are assigned.

Table 4.5: Effect of propagation on the illustrative instance (value assignments tried)

Method	Value assignments tried	Remarks
Plain backtracking	15	Wasted tries on values that could have been pruned (trace above).
Backtracking + forward checking	8	M=1, P=2, A=3, D=2, P=3, A=2, D=3, N=2.
Backtracking + MAC (AC-3)	6	M=1, P=2 (fails at once), P=3, A=2, D=3, N=2.

These counts come from counting the steps of this small example; they only illustrate the trend. In larger instances the saving can be much greater because failures are detected high in the search tree, pruning whole sub-trees. Propagation costs extra computation per step, so the best choice balances propagation effort against the saving.

4.3.4 Task (iv): Real-world challenges and the best strategy for scalability

Table 4.6: Challenges and strategies

Challenge	Why it matters	Strategy
Problem size	Thousands of exams and students; the number of constraints grows with shared students; timetabling is NP-hard.	Good heuristics (MRV, degree, least-constraining), MAC, conflict-directed backjumping and restarts; decompose into smaller sub-problems (by faculty, programme or day).
Possibly no perfect solution	Real data can be over-constrained.	Treat some constraints as soft (weighted CSP), report conflicts, relax the lowest-priority ones.

Challenge	Why it matters	Strategy
Quality, not just feasibility	Students prefer spread-out exams; administrators prefer short exam periods.	After a feasible timetable is found, improve with local search (min-conflicts, tabu search, simulated annealing).
Late changes	Withdrawals, hall changes and staff absence.	Repair the existing timetable by changing as few assignments as possible instead of re-solving from scratch.
Data quality and special needs	Wrong enrolment data or missing accommodations produce invalid timetables.	Validate data, encode accommodations as hard constraints, provide explanations for failures.
Computation time	Administrators need results within hours.	Use anytime behaviour: return the best timetable found so far; parallelise independent sub-problems.

Recommended strategy: a hybrid CSP approach. (1) Validate data and apply preprocessing (node and arc consistency). (2) Use backtracking with MRV, degree and least-constraining-value heuristics and MAC to build a feasible timetable for the hard constraints, decomposing the problem where the conflict graph allows. (3) Optimise the soft constraints with local search. (4) For later changes, apply min-conflicts repair. This keeps the guarantee that hard constraints are respected, while remaining scalable and flexible.

4.4 Conclusion

Exam timetabling fits the CSP framework naturally: courses (slots, halls) are variables, time slots and halls are domains and student, hall, precedence, faculty and accommodation rules are constraints. Backtracking search finds a solution systematically, and constraint propagation (forward checking and arc consistency) with ordering heuristics avoids exploring hopeless branches. For real universities, a hybrid of heuristic backtracking with MAC, problem decomposition and local-search optimisation and repair is the most scalable and practical strategy.

5. SCENARIO 5: STRATEGIC GAME AI USING MINIMAX AND ALPHA-BETA PRUNING

5.1 Scenario Overview

A gaming company is building an AI opponent for a real-time strategy game. The AI competes against human players, must decide within strict time limits and has to evaluate complex game states with many possible moves. The decision tree is extremely large, so computation is expensive.

5.2 Objective

To show how Minimax models the competition, analyse the cost of searching large game trees, explain how Alpha-Beta pruning reduces that cost, design an evaluation function and recommend a strategy that balances decision quality with real-time performance.

5.3 Detailed Answers

5.3.1 Task (i): How Minimax models this problem

Technical explanation. Minimax models a **two-player, zero-sum** game in which the AI (**MAX**) tries to maximise a utility value and the human opponent (**MIN**) tries to minimise it. The game is described by:

- **State *s*:** the positions, health, resources and units of both players.
- **Actions *ACTIONS(s)*:** the legal moves available (move unit, attack, build, research …).
- **Result *RESULT(s, a)*:** the new state after the move.
- **Terminal test:** the game is won/lost, or a depth/time limit is reached.
- **Utility *UTILITY(s)*:** the value of a terminal state for MAX (for example +1 win, 0 draw, −1 loss), or an evaluation-function estimate at a cut-off state.

The Minimax value assumes that **both players play optimally**:

$$MINIMAX(s) = UTILITY(s), \quad \text{if } s \text{ is a terminal state}$$

$$MINIMAX(s) = \max_a MINIMAX(RESULT(s, a)), \quad \text{if it is MAX's turn}$$

$$MINIMAX(s) = \min_a MINIMAX(RESULT(s, a)), \quad \text{if it is MIN's turn}$$

The AI chooses the move that leads to the highest Minimax value, so it plays safely against the best possible human reply. Minimax is explored depth-first, so it needs memory proportional to $b \cdot d$.

Assumptions and limits. A real-time strategy game is not naturally a turn-based, perfect-information game (players act at the same time and may not see each other's units). To apply Minimax, time is divided into decision steps and actions are grouped into a manageable set of abstract moves, and the opponent is assumed to be a reasonable maximiser of its own result. Hidden information is not modelled by plain Minimax.

5.3.2 Task (ii): Computational challenges of large game trees

If the average number of legal moves (branching factor) is b and the search looks d moves ahead, Minimax visits about b^d leaf positions: time $O(b^d)$, memory $O(b \cdot d)$.

Table 5.1: Approximate number of leaf positions b^d searched by plain Minimax

Branching factor b	d = 4	d = 6	d = 8
10	10,000	1,000,000	100,000,000
20	160,000	64,000,000	25,600,000,000
30	810,000	729,000,000	656,100,000,000

Illustration (assumed speed). If the AI could evaluate 1,000,000 positions per second (an assumption used only for illustration), $b = 30$ and $d = 6$ would take about 729 seconds, and $d = 8$ would take about 656,100 seconds (about 7.6 days), far beyond a time limit of a fraction of a second per move.

Other challenges:

- **Costly evaluation:** every leaf needs an evaluation of a complex state (many units and resources).
- **Time limit:** the search must stop at a fixed time, whatever its depth.
- **Horizon effect:** a depth cut-off can hide a disaster just beyond the search depth, or push the AI to delay an unavoidable loss.
- **Repeated states:** different move orders reach the same state and waste effort unless they are stored.
- **Quality of the evaluation function:** a deep search with a poor evaluation can still choose poorly.

5.3.3 Task (iii): How Alpha-Beta pruning improves efficiency

Technical explanation. Alpha-Beta pruning returns exactly the same decision as Minimax but ignores branches that cannot influence it. Two bounds are carried down the tree:

- **α (alpha):** the best (highest) value that MAX can already guarantee along the path to the current node; it starts at −∞.
- **β (beta):** the best (lowest) value that MIN can already guarantee along the path; it starts at +∞.

Pruning rule: if at any node $\alpha \geq \beta$, the remaining children are skipped, because the opponent (or the player above) would never allow the game to reach this node.

```
ALPHA-BETA(s, α, β):
 if s is terminal or depth limit reached: return EVALUATE(s)
 if MAX to move:
   v ← −∞
   for each action a:  v ← max(v, ALPHA-BETA(RESULT(s,a), α, β));  α ← max(α, v)
                if α ≥ β: break          // β cut-off
   return v
 else (MIN to move):
   v ← +∞
   for each action a:  v ← min(v, ALPHA-BETA(RESULT(s,a), α, β));  β ← min(β, v)
                if α ≥ β: break          // α cut-off
   return v
```

Worked example (Figure 5.1). The root A is a MAX node with three MIN children B, C and D, each with three leaf values (evaluation scores from the AI's point of view): B = (3, 12, 8), C = (2, 4, 6), D = (2, 14, 5).

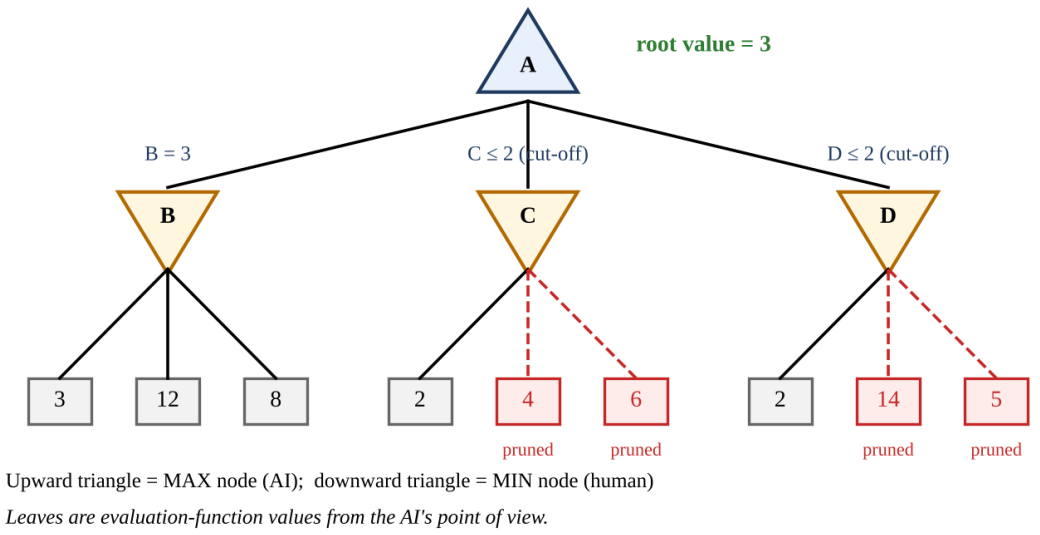


Figure 5.1: Alpha-Beta pruning on a small game tree. Dashed red leaves are never evaluated.

Table 5.2: Step-by-step evaluation

Step	Node / leaf	α	β	Action and reasoning
1	A (root)	−∞	+∞	Start with the widest window; go to B.

Step	Node / leaf	α	β	Action and reasoning
2	B, leaf 3	−∞	+∞	B (MIN) → β = 3.
3	B, leaves 12 and 8	−∞	3	Neither is below 3, so B = 3. Back at A: α = 3 (MAX can already get 3).
4	C, leaf 2	3	+∞	C (MIN) can give at most 2, and 2 ≤ α = 3.
5	C, leaves 4 and 6	3	2	**Pruned**: C cannot be better than 2, so MAX will never choose C over B.
6	D, leaf 2	3	+∞	D can give at most 2, and 2 ≤ α = 3.
7	D, leaves 14 and 5	3	2	**Pruned** for the same reason.
8	A (root)	3	+∞	Root value = max(3, ≤2, ≤2) = **3**; best move is A → B.

Only 5 of the 9 leaves (3, 12, 8, 2, 2) were evaluated, yet the decision (value 3, move to B) is exactly what plain Minimax gives. The values returned for C and D are only upper bounds (≤ 2), which is sufficient for the decision.

- **Best case (perfect move ordering):** about $O(b^{d/2})$ positions, so the search can look about **twice as deep** in the same time. For b = 30 and d = 6: 30^3 = 27,000 instead of 729,000,000 (the same illustrative speed gives 0.027 s instead of 729 s).
- **Worst case (poor ordering):** $O(b^d)$, no saving, although the result is still correct.
- **Move ordering matters:** examine promising moves first (captures, attacks, the best move from the previous search).

5.3.4 Task (iv): Design of an evaluation function and justification of its factors

Since the search cannot reach the end of the game, it is cut off at a depth limit and the state is scored by an evaluation function that estimates the chance of winning. A common design is a weighted linear sum of normalised features:

$$E(s) = w_1 f_1(s) + w_2 f_2(s) + w_3 f_3(s) + w_4 f_4(s) + w_5 f_5(s)$$

$$f_i(s) = (X_i^{AI} - X_i^{H}) / (X_i^{AI} + X_i^{H}), \quad -1 \le f_i \le 1$$

Here X_i^{AI} and X_i^{H} are the values of quantity i for the AI and the human; f_i is positive when the AI is ahead (and 0 if both are 0); w_i is the weight of feature i, with $w_1 + \ldots + w_5 = 1$, so $E(s)$ lies between −1 and +1. If the game is already won or lost, $E(s)$ is replaced by +1 or −1 (or a very large number).

Table 5.3: Features, weights and justification (initial weights, to be tuned)

Feature	Definition of X (AI vs human)	Weight	Justification
f_1 Military strength	Total strength of units: health × attack of army	0.30	The army decides battles directly, so it is the strongest short-term indicator of advantage.

Feature	Definition of X (AI vs human)	Weight	Justification
f_2 Economy / resources	Resource stock + income rate + number of workers	0.25	Resources create future units and upgrades; a strong economy converts into long-term advantage.
f_3 Territory / map control	Controlled area, key positions, vision	0.20	Control gives access to resources, safe movement and information.
f_4 Base safety	Health of buildings and defences	0.15	Losing the base normally loses the game, so threats to it must be reflected.
f_5 Technology	Upgrades and unlocked unit types	0.10	Improves future strength, but its effect is slower and less certain.

Numerical example. Suppose the AI/human values are: military 60/40, economy 90/110, territory 65/35, base safety 55/45 and technology equal. Then $f_1 = 0.20$, $f_2 = -0.10$, $f_3 = 0.30$, $f_4 = 0.10$, $f_5 = 0$. $E(s) = 0.30(0.20) + 0.25(-0.10) + 0.20(0.30) + 0.15(0.10) + 0.10(0) = 0.06 - 0.025 + 0.06 + 0.015 + 0 = 0.11$. The positive value shows that the AI is slightly ahead.

Design principles: the function must be fast to compute (it is called at every leaf), correlated with the real chance of winning, bounded and consistent with terminal values. The weights above are starting values chosen by reasoning about the game; they should be tuned by play-testing, self-play or fitting to game data, and different weights can be used in the opening, middle and end of a game.

5.3.5 Task (v): Strategy for balancing optimality and real-time performance

Full optimality is impossible in a real-time game, so the aim is the best decision *within the time budget*. The recommended strategy combines the following techniques:

- **Iterative deepening with a time limit:** search depth 1, 2, 3 … until time is nearly over; always keep the best move of the last *completed* depth, so a move is available at any moment (anytime behaviour).
- **Alpha-Beta pruning with good move ordering:** use the best move of the previous iteration first, plus attack/capture moves, to obtain pruning close to the best case.
- **Transposition table:** store values of positions already evaluated so that repeated states are not searched again.
- **Action abstraction and move pruning:** group unit actions into a small set of strategic macro-moves and discard obviously bad moves to reduce *b*.
- **Selective deepening (quiescence):** continue the search in unstable positions, such as ongoing fights, to reduce the horizon effect.

- **Cheap but informative evaluation function:** compute features incrementally; update them when moves are made instead of recomputing.
- **Dynamic time management:** give more time to critical moments and less to routine moves.
- **Alternatives if *b* remains very large:** sample-based methods such as Monte Carlo Tree Search can be combined with, or replace, Minimax.

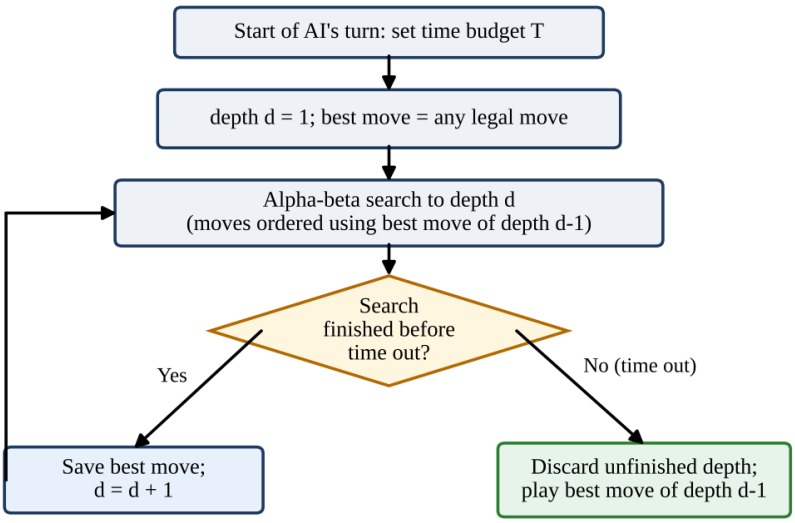


Figure 5.2: Iterative-deepening Alpha-Beta search under a time limit.

Table 5.4: Trade-off between quality and speed

Setting	Decision quality	Computation cost	Use when
Shallow depth, strong pruning	Lower (may miss long plans)	Very low	Time is very short or many units on the map.
Deep search, good ordering	Higher	Medium	Normal situations with moderate branching.
Plain Minimax	Same as Alpha-Beta at equal depth	Very high	Not suitable for real time.
Alpha-Beta + iterative deepening + transposition table	Best available within the time budget	Controlled by the time limit	**Recommended default.**

5.4 Conclusion

Minimax lets the AI choose a move that is best against the strongest human reply, but its cost, $O(b^d)$, is too high for real-time play. Alpha-Beta pruning gives the same decision while skipping irrelevant branches, and in the best case it allows about double the search depth. A weighted evaluation function based on military strength, economy, territory, base safety and technology

scores the cut-off positions. Using iterative deepening, move ordering, transposition tables and abstraction under a strict time limit is the recommended strategy: it does not guarantee a perfect move, but it gives the best move that can be found within the time available.

6. OVERALL CONCLUSION AND REFERENCES

6.1 Overall Conclusion

The five scenarios show that the choice of an AI search technique depends on the structure of the problem: whether the path or only the final configuration matters, whether the environment is known, whether the problem has hard constraints, and whether an opponent is involved. The recommendations are summarised below.

Table 6.1: Summary of recommended techniques

Scenario	Problem type	Recommended technique	Main reason
1. Delivery drones	Shortest/safest path, dynamic costs	A* (weather-aware admissible heuristic) with re-planning; Weighted A* if time is critical	Includes real costs and risk, optimal with admissible *h*, adapts to blocked paths.
2. Traffic signals	Optimization with huge search space	Hybrid Genetic Algorithm + Simulated Annealing by zones	Escapes local maxima, scalable (parallel, zones), adaptable (warm start).
3. Mars rover	Unknown, partially observable environment	Online replanning agent (probabilistic map, A*/D* Lite, LRTA* learning, safety layer)	Acts with incomplete information and stays safe.
4. Exam timetable	Constraint satisfaction	Backtracking with MRV/degree/LCV + MAC, decomposition, local-search optimisation and repair	Guarantees hard constraints, scales through pruning and decomposition.
5. Strategy game AI	Adversarial search	Alpha-Beta with iterative deepening, move ordering, transposition table, weighted evaluation	Same decision as Minimax at lower cost; best move within the time limit.

A common lesson is that guaranteed optimality and practical performance are different things. A*'s optimality depends on an admissible heuristic, local-search methods give good but unproven solutions, online search trades optimality for safety and adaptability, and depth-limited game search is optimal only with respect to the evaluation function at the depth reached. Stating these assumptions clearly is part of using each technique correctly.

6.2 References

The concepts in this assignment are standard material in artificial intelligence. The following are the standard sources on which these topics are based:

1. Russell, S. and Norvig, P. *Artificial Intelligence: A Modern Approach*, 4th edition. Pearson, 2021. (Chapters on search, local search, online search, constraint satisfaction problems and adversarial search.)

2. Hart, P. E., Nilsson, N. J. and Raphael, B. “A formal basis for the heuristic determination of minimum cost paths.” *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 1968, pp. 100–107. (A* search.)
3. Knuth, D. E. and Moore, R. W. “An analysis of alpha-beta pruning.” *Artificial Intelligence*, 6(4), 1975, pp. 293–326.
4. Kirkpatrick, S., Gelatt, C. D. and Vecchi, M. P. “Optimization by simulated annealing.” *Science*, 220(4598), 1983, pp. 671–680.
5. Holland, J. H. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975. (Genetic algorithms.)
6. Korf, R. E. “Real-time heuristic search.” *Artificial Intelligence*, 42(2–3), 1990, pp. 189–211. (LRTA*.)
7. Koenig, S. and Likhachev, M. “D* Lite.” *Proceedings of the AAAI Conference on Artificial Intelligence*, 2002. (Incremental re-planning.)